

Relay: Graph-Memory Agent Team

Project Requirements and Build Plan for Graph Hacks (WeMakeDevs x FalkorDB)

Track: Agent Memory and Coordination | **Mode:** Solo | **Level:** Intermediate (Python) **Event:** Oct 15 to Oct 18, 2026 (online) | **Prep window:** Oct 5 to Oct 14

1. Hackathon Rules Summary

- Build a working agent (or group of agents) with **FalkorDB as the context layer**. The graph must shape what the agent does, not sit on the side.
- Solo or team of up to 4. You are entering solo.
- Submissions open Oct 15, 12:01 AM (GMT+5:30). You cannot change your team after submitting.
- Prizes: one per track, a best blog post prize (AirPods 5), and t-shirts for the top 50 teams.
- **Action item:** open the Rules tab on the event page and confirm submission format (repo link, demo video, etc.) before Oct 15. This document does not assume it.

Track 02 key focus (your judging criteria):

Criterion	How Relay meets it
Episodic, semantic, procedural memory	Three memory types stored as distinct node labels
Persistent conversation history	Every conversation saved as graph nodes
Shared state and handoffs between agents	Agents pass tickets through the graph
High-concurrency reads and writes	Load test script with parallel sessions
Separate memory per user or tenant	One FalkorDB graph per tenant

2. Problem Statement

Problem: AI support agents forget. A customer explains an issue on Monday, and on Thursday a different agent (or the same one in a new session) asks them to start over. Agents also cannot share what they learn, and they cannot explain why they gave an answer.

Solution: *Relay* is a small team of cooperating agents that keep all their memory in a FalkorDB graph. Every customer interaction becomes connected data. Any agent can recall past sessions, learn which fixes work, hand a case to another agent, and show the path in the graph behind each answer.

Why a graph: The key questions are about connections: "Has this customer had this problem before?", "Which fix worked for similar problems?", "Which agent handled this last time?". A graph answers these in one traversal.

3. Goals and Non-Goals

Goals

- 1. Three agents that collaborate through a shared graph.
- 2. Memory that persists across sessions and updates over time.
- 3. Strict isolation: each tenant has its own graph.
- 4. An "Explain" view that shows the graph path behind every answer.
- 5. A clean demo, a README, and a blog post.

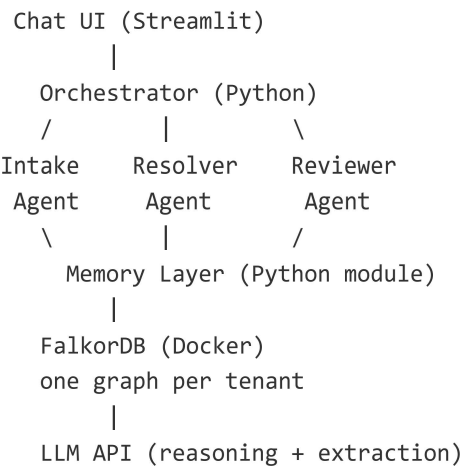
Non-goals (do not build these)

- Real authentication, billing, or production deployment.
- Voice, email, or chat-platform integrations.
- Training or fine-tuning any model.

4. Users and Use Case

- **End user (demo persona):** a customer chatting with support.
- **Operator (demo persona):** a support lead who watches the graph and the Explain panel.
- **Demo scenario:** three tenants (companies) each with a few customers. A returning customer reports a problem they had last month. The agents recall it, apply a fix that worked before, and the Explain panel shows the path.

5. System Architecture



Agents

Agent	Job	Reads from graph	Writes to graph
Intake	Understands the message, extracts entities (product, issue type, urgency), recalls customer history	Customer, past Episodes	New Episode, Issue
Resolver	Finds the best fix using similar past issues and playbooks	Issues, Solutions, Playbooks	Attempted Solution, outcome
Reviewer	Checks for quality of the solution and updates the graph	Customer, Solution, Outcome	Updated Solution, Outcome

reviewer	Checks the outcome, updates playbook success counts, flags handoffs	Solutions, outcomes	Updated Playbook stats, Handoff
-----------------	---	---------------------	---------------------------------

The agents never talk to each other directly. They coordinate by reading and writing the graph (a **blackboard** pattern). This is what makes shared state and handoffs visible and traceable.

6. Graph Data Model

Node labels

Label	Memory type	Key properties
Customer	Semantic	id, name, plan
Product	Semantic	name, version
Issue	Semantic	type, summary, status
Episode	Episodic	id, timestamp, transcript_summary, session_id
Message	Episodic	role, text, timestamp
Solution	Semantic	description
Playbook	Procedural	name, steps, success_count, fail_count
Agent	Coordination	name, role
Handoff	Coordination	reason, timestamp, status

Relationships

```

(Customer)-[:HAD_EPISODE]->(Episode)
(Episode)-[:HAS_MESSAGE]->(Message)
(Episode)-[:ABOUT]->(Issue)
(Issue)-[:AFFECTS]->(Product)
(Issue)-[:SIMILAR_TO]->(Issue)
(Solution)-[:RESOLVED]->(Issue)
(Playbook)-[:APPLIES_TO]->(Issue)
(Playbook)-[:INCLUDES]->(Solution)
(Agent)-[:HANDLED]->(Episode)
(Agent)-[:HANDED_OFF_TO]->(Agent)    // via Handoff node

```

Tenant isolation

Create one graph per tenant, for example `tenant_acme` and `tenant_globex`. Select the graph by tenant ID on every call. No query can cross tenants because the graph itself is different.

Example Cypher queries

Recall a customer's past issues:

```

MATCH (c:Customer {id:$cid})-[:HAD_EPISODE]->(e:Episode)-[:ABOUT]->(i:Issue)
RETURN i.type, i.summary, e.timestamp
ORDER BY e.timestamp DESC LIMIT 5

```

Best fix for an issue type (procedural memory):

```
MATCH (p:Playbook)-[:APPLIES_TO]->(i:Issue {type:$type})
RETURN p.name, p.steps, p.success_count, p.fail_count
ORDER BY p.success_count DESC LIMIT 3
```

Prior solutions that worked for similar issues:

```
MATCH (i:Issue {type:$type})<-[:SIMILAR_TO*0..2]-(j:Issue)<-[:RESOLVED]-(s:Solution)
RETURN s.description, count(*) AS times
ORDER BY times DESC LIMIT 3
```

7. Functional Requirements

ID	Requirement	Priority
FR1	User can chat with Relay by selecting a tenant and customer	Must
FR2	Intake agent extracts issue type and product from messages	Must
FR3	System saves every conversation as Episode and Message nodes	Must
FR4	On a returning customer, agents recall past episodes in the reply	Must
FR5	Resolver suggests a fix using playbooks and past solutions	Must
FR6	Reviewer records outcome and updates playbook success or fail counts	Must
FR7	Handoff between agents is recorded as a node and visible in the UI	Must
FR8	Each tenant uses its own graph, with no data crossing	Must
FR9	Explain panel shows the graph path and Cypher used for the answer	Must
FR10	Graph visualization panel for the current customer	Should
FR11	Seed script loads demo data for 2 to 3 tenants	Must
FR12	Load test with parallel sessions, with results in the README	Should
FR13	Forget or edit a memory (delete an Episode)	Could

8. Non-Functional Requirements

- **Traceability:** every answer links to graph nodes or edges used.
- **Concurrency:** handle at least 20 parallel sessions in the load test without data errors.
- **Safety:** use parameterized Cypher queries only. Never build queries by string concatenation with user text.
- **Setup:** one command to start (`docker compose up`) and one to seed data.
- **Cost:** keep LLM calls small (short prompts, one extraction call per message).

9. Tech Stack

Layer	Choice	Why
Graph DB	FalkorDB (Docker)	Required context layer; one graph per tenant
Language	Python 3.11+	Your comfort level, best FalkorDB support
DB client	<code>falkordb</code> Python package	Official client
LLM	An LLM API such as Claude (via the <code>anthropic</code> SDK)	Reasoning and entity extraction
Backend	FastAPI	Simple API for the orchestrator
UI	Streamlit	Fastest way to a clean demo UI
Graph view	FalkorDB Browser for the demo, or a small <code>pyvis</code> graph in Streamlit	Show relationships visually
Testing	<code>pytest</code> , plus an <code>asyncio</code> load script	Proves concurrency
Optional	MCP server exposing the memory tools; Graphiti	Stretch goals only

Decision rule: build the memory layer yourself first. It is small, you will understand it fully, and you can explain it in the blog. Add Graphiti or MCP only if everything else is done.

Project structure

```
relay/  
  docker-compose.yml  
  app/  
    main.py          # FastAPI app  
    orchestrator.py  # runs Intake -> Resolver -> Reviewer  
    agents/  
      intake.py  
      resolver.py  
      reviewer.py  
    memory/  
      graph.py       # FalkorDB connection, tenant selection  
      queries.py     # all Cypher queries  
      llm.py         # LLM wrapper  
  ui/streamlit_app.py  
  scripts/seed.py  
  scripts/load_test.py  
  tests/  
  README.md
```

10. How to Build It: Step by Step

Phase 0: Setup (Oct 5 to 6)

1. Install Docker, Python, and an editor. Get an LLM API key.
2. Run `docker pull falkordb/falkordb:latest`, then start it and open FalkorDB Browser.
3. Join the FalkorDB Discord.
4. Create the GitHub repo and folder structure above.

Phase 1: Graph basics (Oct 7 to 9)

1. In Python, connect to FalkorDB, create a graph, add a few nodes, run a query.
2. Write `graph.py` with a `get_graph(tenant_id)` function.
3. Write the schema from section 6 and the first three queries.
4. Write `seed.py` to load demo tenants, customers, issues, and playbooks.

Phase 2: Agents (Oct 10 to 12)

1. Write `llm.py` (one function that takes a prompt and returns text or JSON).
2. Build the **Intake** agent: LLM extracts `{issue_type, product, urgency}` as JSON, then saves an Episode.
3. Build the **Resolver**: runs the playbook query, passes results to the LLM, and gets a reply.
4. Build the **Reviewer**: records the outcome and updates playbook counts.
5. Connect them in `orchestrator.py`.

Phase 3: UI (Oct 13 to 14)

1. Streamlit page with tenant picker, customer picker, and chat box.
2. Side panel: **Explain** (Cypher run, nodes touched, agent that acted).
3. Dry run the whole demo twice. Fix bugs. Write a first README draft.

Hackathon days

Day	Focus
Oct 15	Submissions open. Finish any Must requirements still open. Verify tenant isolation with a test.
Oct 16	Handoff records, Explain panel polish, graph visualization, procedural memory updates
Oct 17	Load test, README, architecture diagram, demo video recording, blog draft
Oct 18	Freeze code in the morning. Publish blog, record final demo, submit well before 11:59 PM GMT+5:30

11. Making It Stand Out (Efficiency and Quality)

1. **Memory that learns.** Show a playbook's success count change after a resolved case. This is the strongest proof of procedural memory.
2. **Traceable answers.** Every reply shows "I Used: Episode 12 Playbook Reset Password (8 wins, 1 fail)"

2. **Traceable answers.** Every reply shows "Good episode 12, Playbook next resolved (0 times) relay". Judges asked for explainability.
 3. **Tenant isolation proof.** A test that tries to read tenant B from tenant A and fails.
 4. **Concurrency numbers.** A single table in the README: sessions, average latency, errors.
 5. **Efficiency tips.**
 - Add indexes on the properties you match on (`Customer.id`, `Issue.type`).
 - Keep prompts short: send only the top 3 graph results to the LLM.
 - Cache the playbook query per issue type within a session.
 - Use one LLM call for entity extraction, not several.
 6. **Tell the story.** Open the demo with the "start over" pain, then show Relay remembering.
-

12. Testing Plan

Test	Pass condition
Unit: queries	Each query returns expected nodes on seed data
Memory persistence	Restart the app, ask again, history still recalled
Isolation	Tenant A cannot see tenant B's customer
Handoff	Handoff node exists after an escalation
Playbook learning	Success count increases after a resolved case
Load	20 parallel sessions finish with no errors
Injection safety	Odd characters in messages do not break queries

13. Risks and Fixes

Risk	Fix
Scope too big for solo	Only the Must items count. Cut Should and Could first.
LLM returns bad JSON	Ask for JSON only, parse safely, retry once with a fallback
Demo breaks live	Record a backup video. Seed data makes the demo repeatable.
Time runs out	Freeze features on Oct 17 evening
Using AI code you don't understand	Read and run every piece. You must be able to explain it.

14. Using AI Tools to Build It

- **Claude or a coding assistant:** ask for one file at a time ("write `graph.py` with these functions"), then run it. Paste errors back in.
- **Good prompt pattern:** give the schema from section 6, the file's purpose, and the exact function

Good prompt pattern: give the schema from section 9, the file's purpose, and the exact function signatures you want.

- **Ask for tests** after each module.
 - **Use AI for the blog and README drafts**, then rewrite in your own voice with your real numbers.
 - **Never paste API keys** into prompts. Keep them in a `.env` file excluded from git.
-

15. Deliverables Checklist

- ☐ Public GitHub repo with README (setup, architecture, data model)
 - ☐ Working demo (3 tenants, returning customer scenario)
 - ☐ Explain panel and graph view
 - ☐ Load test results
 - ☐ Demo video (2 to 3 minutes)
 - ☐ Blog post (for the side prize): problem, why a graph, schema, what you learned
 - ☐ Submission completed before the deadline, using the format on the Rules tab
-

16. Demo Script (3 minutes)

1. **Problem (20s):** "Agents forget. Customers repeat themselves."
 2. **Customer returns (40s):** Relay greets with the past issue.
 3. **Agents at work (60s):** Intake, Resolver, and Reviewer each act. Show the handoff.
 4. **Explain panel (30s):** Show the graph path and Cypher.
 5. **Learning (20s):** Playbook success count goes up.
 6. **Isolation and scale (10s):** Show tenant graphs and load test numbers.
-

17. Glossary

- **Node / edge:** a thing and a link between things in a graph.
- **Cypher:** the query language for graphs.
- **Episodic memory:** what happened (past conversations).
- **Semantic memory:** what is known (customers, products, facts).
- **Procedural memory:** how to do things (playbooks that improve).
- **Blackboard pattern:** agents coordinate by reading and writing a shared store.
- **MCP:** Model Context Protocol, a standard way to give agents tools.

